

Resumen

Para reservar memoria:

L = (TLista)malloc(sizeof(TNodo))

Archivos

FILE *arch

arch = fopen("Nombre", "función + tipo")

funcion:

- "w": write
- "r": read

tipo:

- "t": texto
- "b": binario

Para .txt:

- fprintf(arch, cadena de control, arg1, arg2, ... , argn)
- fscanf(arch, cadena de control, arg1, arg2, ... , argn)

Para .dat:

- fwrite(*ptr, tamaño, n, arch)
- fread(*ptr, tamaño, n, arch)

*ptr es el puntero a donde se almacena o de donde se guarda la información, n es la cantidad de elementos a guardar/leer

TDA Pila

inicia(P) → Devuelve en P una pila vacía

vacíaP(P) → Devuelve Verdadero si la pila P está vacía, y Falso en caso contrario

consultaP(P) → Devuelve el valor del elemento que se encuentra en el tope de la pila P

sacaP(P , x) → Devuelve en x el elemento que se encuentra en el tope de la pila removiéndolo de P

poneP(P , x) → Inserta el elemento x en el tope de la pila.

TDA Cola

iniciaC(C) → Devuelve en C una cola vacía

vacíaC(C) → Devuelve Verdadero si la cola C está vacía, y Falso en caso contrario

consultaC(C) → Devuelve el valor del elemento que se encuentra en el primer lugar de la cola C

sacaC(C , x) → Devuelve en x el elemento que se encuentra en el primer lugar de la cola removiéndolo de C

poneC(C , x) → Inserta el elemento x en el último lugar de la cola.

Lista simplemente enlazada

```
typedef struct nodo {  
    int dato;  
    struct nodo *sig;  
} TNode;  
typedef TNode *TLista;
```

Cada nodo conoce solamente al siguiente. Para insertar y eliminar se usan dos punteros (ant, act)

Casos de Inserción:

- Inserto en la cabeza
- Inserto en el medio

Casos de Eliminación:

- Elimino la cabeza
- Elimino en el medio

Lista doblemente enlazada

```
typedef struct nodo {  
    int dato;  
    struct nodo *sig, *ant;  
} TNodeD;  
typedef TNodeD *PnodeD;  
typedef struct {  
    PnodeD pri, ult;  
} TListaD;
```

Cada nodo tiene puntero al siguiente y al anterior. Para insertar y eliminar se usan un solo puntero

Casos de Inserción:

- Inserto en el principio
 - Está vacía
 - No está vacía
- Inserto en el medio
- Inserto al final

Casos de Eliminación:

- Elimino el del principio
 - Es único
 - No es único
- Elimino en el medio
- Elimino en el final

Lista circular

```
typedef struct nodo {  
    int dato;  
    struct nodo *sig;  
} TNodo;  
typedef TNodo *TListaC;
```

```
if (L != NULL) {  
    aux = L;  
    do {  
        printf("%d ", aux->dato);  
        aux = aux->sig;  
    } while (aux != L);  
}
```

Si están ordenados, LC almacena la dirección del último nodo (el más grande)

Casos de Inserción:

- Inserto al principio
 - Lista vacía
 - Lista no vacía
- Inserto en el medio

Casos de Eliminación:

- Elimino el del principio
 - Es único
 - No es único
- Elimino en el medio

```
if (*L != NULL) {  
    act = *L;  
    ant = NULL;  
    do {  
        ant = act;
```

```

        act = act->sig;
    } while (act != *L && act->dato != x);
}

```

Arboles Binarios

```

typedef struct nodoA {
    int dato;
    struct nodoA *izq, *der;
} TNodeA;
typedef TNodeA *TArbol;

```

```

void PreOrden(TArbol A) {
    if (A != NULL) {
        printf("%d ", A->dato);
        PreOrden(A->izq);
        PreOrden(A->der);
    }
}

```

```

void InOrden(TArbol A) {
    if (A != NULL) {
        InOrden(A->izq);
        printf("%d ", A->dato);
        InOrden(A->der);
    }
}

```

```

void PosOrden(TArbol A) {
    if (A != NULL) {
        PosOrden(A->izq);
        PosOrden(A->der);
        printf("%d ", A->dato);
    }
}

```

```

int Altura(TArbol A) {
    int hi, hd;
    if (A == NULL)
        return 0;
    else {
        hi = Altura(A->izq);

```

```

    hd = Altura(A->der);
    if (hi > hd)
        return 1 + hi;
    else
        return 1 + hd;
}
}

```

Árbol Binario de Búsqueda - ABB

Regla: izquierda < raíz < derecha

```

TArbol BuscaABB(TArbol A, int x) {
    if (A == NULL)
        return NULL;
    else
        if (A->dato == x)
            return A;
        else
            if (x < A->dato)
                return BuscaABB(A->izq, x);
            else
                return BuscaABB(A->der, x);
}

```

```

void InsertaABB(TArbol *A, int x) {
    if (*A == NULL) {
        *A = malloc(sizeof(TNodoA));
        (*A)->dato = x;
        (*A)->izq = NULL;
        (*A)->der = NULL;
    }
    else
        if (x < (*A)->dato) {
            InsertaABB(&(*A)->izq, x);
        }
        else
            if (x > (*A)->dato) {
                InsertaABB(&(*A)->der, x);
            }
}
}

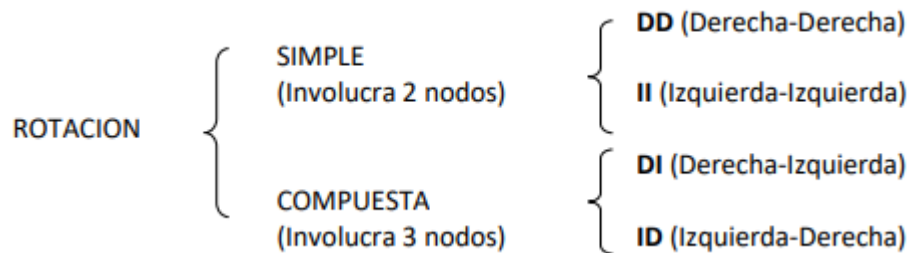
```

Árboles Balanceados por su Altura (AVL)

Regla: Es un ABB donde para todo nodo del árbol, la diferencia entre la altura de sus subárboles izquierdo y derecho es a lo sumo 1.

Factor de Equilibrio: $FE = h_i - h_d$

Rotaciones



Una vez que se detecta un nodo para el cual el factor de equilibrio es 2 o -2, se debe avanzar (a partir de éste) dos niveles por la rama más larga. El camino que se sigue determina la rotación.

Algoritmos de Rotación

☛ Rotación DD DERECHA(Nodo) $\leftarrow$ IZQUIERDA(Nodo1) IZQUIERDA(Nodo1) $\leftarrow$ Nodo Nodo $\leftarrow$ Nodo1 ☛ Rotación II IZQUIERDA(Nodo) $\leftarrow$ DERECHA(Nodo1) DERECHA(Nodo1) $\leftarrow$ Nodo Nodo $\leftarrow$ Nodo1	☛ Rotación DI IZQUIERDA(Nodo1) $\leftarrow$ DERECHA(Nodo2) DERECHA(Nodo2) $\leftarrow$ Nodo1 DERECHA(Nodo) $\leftarrow$ IZQUIERDA(Nodo2) IZQUIERDA(Nodo2) $\leftarrow$ Nodo Nodo $\leftarrow$ Nodo2 ☛ Rotación ID DERECHA(Nodo1) $\leftarrow$ IZQUIERDA(Nodo2) IZQUIERDA(Nodo2) $\leftarrow$ Nodo1 IZQUIERDA(Nodo) $\leftarrow$ DERECHA(Nodo2) DERECHA(Nodo2) $\leftarrow$ Nodo Nodo $\leftarrow$ Nodo2
---	---

En caso de necesitar rebalancear el árbol: En inserción con una rotación ya se estabiliza, en eliminación se puede provocar más de una rotación.

TDA N-ARIO

Sean A variable de tipo árbol general y p es variable de tipo posición:

Vacio(A) -> Devuelve verdadero si A es árbol Vacío.

Nulo(p) -> Devuelve verdadero si p es la posición Nula

HijoMasIzq(p,A) -> Devuelve la posición del hijo más a la izquierda de p, si p es hoja devuelve una posición nula.

HermanoDer(p,A) -> Devuelve la posición del hermano a la derecha de p (tiene el mismo padre de p), si p es el de la extrema derecha devuelve una posición nula.

Info(p,A) -> Devuelve el dato del en la posición p en el árbol A.

Raiz(A) -> Devuelve una posición que es la raíz del árbol A.

Padre(p,A) -> Devuelve la posición del padre de la posición p en el árbol A, si p es la raíz devuelve una posición nula.

Grafos

📌 REPRESENTACION EN MEMORIA de GRAFOS

① Representación Matricial	⇒	Matriz de Adyacencia
② Representación por medio de Listas	{ ⇒	Lista de Adyacencia
	⇒	Multilista

📌 REPRESENTACION EN MEMORIA de DIGRAFOS

① Representación Matricial	⇒	Matriz de Adyacencia
② Representación por medio de Listas	⇒	Lista de Adyacencia

Subgrafos

Sea $G = (V, E)$, un **subgrafo** G' de G es un grafo $G' = (V', E')$ tal que:

- $V' \subseteq V$ y $E' \subseteq E$, tal que en E' hay aristas $(v, w) \in E$ con v y $w \in V'$.

Si en E' están todas las aristas (v, w) de E con v y w en V' , se dice que G' es un **subgrafo inducido**.

Árbol Binario que proviene de una transformación

- Unir hermanos
- Cortar enlace de cada nodo con sus hijos, excepto el de más a la izquierda
- Rotar el árbol levemente en sentido horario (la raíz es la misma)
- En el árbol binario final, su hijo de la izquierda representaba un hijo suyo, pero su hijo de la derecha representaba un hermano suyo

Si proviene de la transformación de un bosque, las raíces de cada árbol están conectadas como “hermanos”

Recorridos

Amplitud (BFS)

- Se selecciona un nodo y se lo visita
- Se visitan todos los adyacentes no visitados

- Se repite con los vértices visitados

Se usa una cola (donde guardamos los vértices a analizar) y un vector de visitados (VV)

Si es no conexo, cada vez que la cola queda vacía es porque terminamos de recorrer esa componente conexa (cc)

- Mientras no estén todos visitados
- cc++
- Seleccionar vértice
- PoneC(C,v)
- Mientras no estén todos visitados y la cola no este vacia
 - Busco adyacentes de v, los visito y los agrego a la cola

Profundidad (DFS)

- Se selecciona un nodo inicial y se lo visita
- Se visita un nodo adyacente y se lo visita
- Se continúa tomando un nodo adyacente al último nodo visitado, en caso de no haber ninguno, se va intentando con los ya visitados en orden inverso al que se los visitó

El recorrido puede ser un camino si nunca se volvió para atrás.

Usamos una pila (donde guardamos los vértices a analizar) y un vector de visitados (VV)

Si es no conexo, cada vez que la pila queda vacía es porque terminamos de recorrer esa componente conexa (cc)

- Mientras no estén todos visitados
- cc++
- Seleccionar vértice
- PoneP(P,v)
- Mientras no estén todos visitados y la pila no este vacia
 - Consulta(P,v)
 - Busco un adyacente a v no visitado
 - Si encuentro uno, lo pongo en P y lo visito
 - Si no lo encuentro, Saca(P,v)

Árboles Abarcadores de Costo Mínimo (AAM)

Un árbol abarcador para G es un árbol libre (grafo conexo acíclico) que conecta todos los vértices de V , su costo es la suma de los costos de las aristas del árbol. Un AAM para G es un árbol abarcador que conecta todos los nodos de V con mínimo costo.

Algoritmo de Kruskal

- Genera un grafo $T = (V, \emptyset)$
- Se examinan las aristas de E en orden creciente de costo. Se agrega la arista si conecta dos vértices que se encuentran en componentes conexas distintas
- Se repite hasta que todos los vértices de V estén en una misma componente conexa.

Se usa un vector CC donde cada posición tiene el número de componente conexa al que pertenece el vértice.

Algoritmo de Prim

- Se inicia con un conjunto U con un vértice cualquiera de V
- Se analizan las aristas que conectan un vértice de U con uno de $V-U$, de ellas se elige la de menor costo y se agrega el vértice de $V-U$ a U
- Se repite hasta que todos los vértices de V estén en U

Caminos más cortos en Digrafos

Algoritmo de Dijkstra (caminos mínimos con un solo origen)

Se definen:

- S como el conjunto de vértices cuya distancia más corta desde el origen ya se conoce
- w como un vértice cuya distancia más corta desde el origen no se conoce
- D como un vector con el costo mínimo conocido desde el origen hasta ese vértice

- P como un vector de predecesores para el camino desde el origen a ese vértice.

Pasos:

- Incluir el origen en S
- Calcular distancias $D[w]$ desde el origen hasta cada uno de los vértices que no se encuentran en S
- Elegir el vértice tal que $D[w]$ sea mínima
- Agregar w a S
- Repetir hasta que todos los vértices de V se encuentren en S

Algoritmo de Floyd (caminos mínimos entre todos los pares de vértices)

Cargar matriz A de costos

```
for (k = 1; k <= N ; k++)
    for (i = 1; i <= N ; i++)
        for (j = 1; j <= N ; j++)
            if (A[i,k] + A[k,j] < A[i,j])
                A[i,j] = A[i,k] + A[k,j];
```

k representa el vértice intermedio

i representa el vértice de origen

j representa el vértice de destino

“Si ir de i a k y después de k a j tiene menor costo que ir de i a j como lo conocía, actualizo el costo mínimo”

Algoritmo de Warshall (existencia de caminos entre todos los pares de vértices)

Cargar matriz A booleana

```
for (k = 1; k <= N ; k++)
    for (i = 1; i <= N ; i++)
        for (j = 1; j <= N ; j++)
            if (A[i,k] + A[k,j] != 0 && A[i,k] + A[k,j] != infinito)
                A[i,j] = 1;
```

k representa el vértice intermedio

i representa el vértice de origen

j representa el vértice de destino

“Si puedo ir de i a k y después de k a j , entonces puedo ir de i a j ”